Atividade Parte - 3

Alunos: Alan Lino Reis, Hilário Canci Neto.
Link Github: https://github.com/alanlinoreis/Lavacar-C

1. Nome e descrição da ferramenta

Nome: Reviewable
Descrição:
O Reviewable é uma ferramenta de revisão de código integrada ao GitHub que melhora a análise de Pull Requests. Ele organiza comentários, destaca mudanças e facilita a colaboração entre revisores, tornando o processo mais eficiente e rastreável.

2. Como acessar ou instalar

Pré-requisitos

- Conta no GitHub
- Repositório com permissões de acesso

Acesso

1. Acesse: https://reviewable.io
2. Clique em **Sign in with GitHub**
3. Autorize o acesso à sua conta

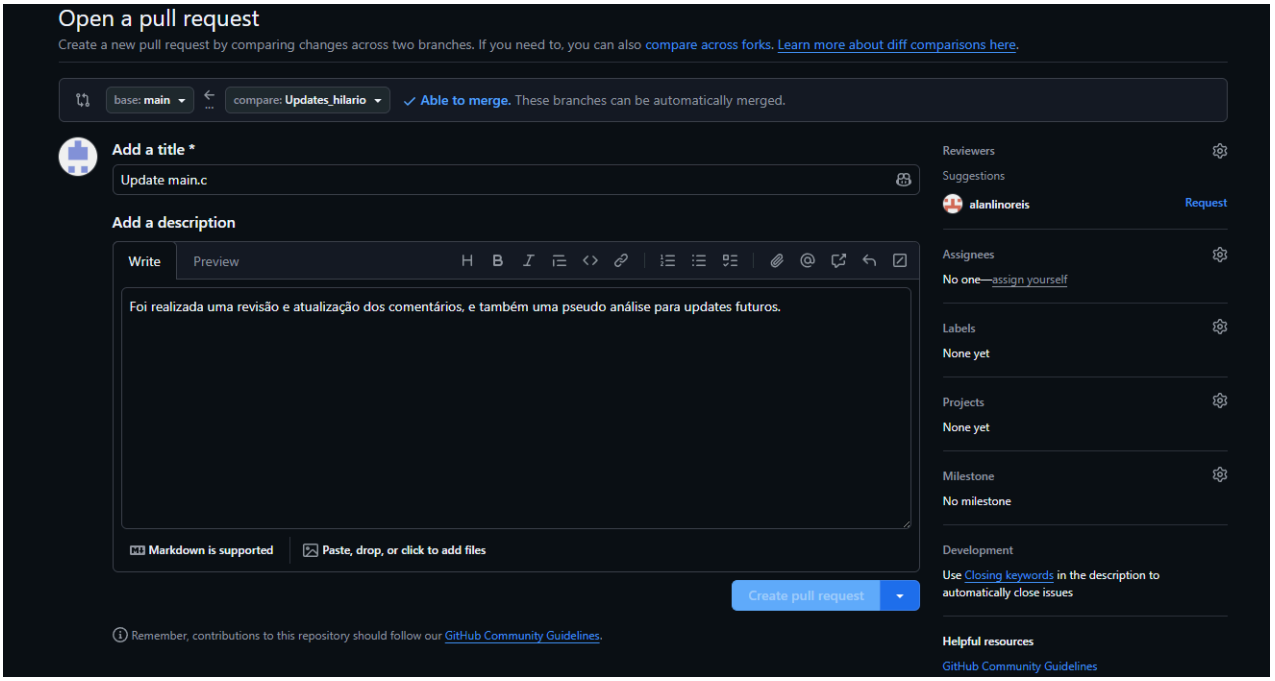
Instalação no repositório

1. Entre no seu repositório no GitHub
2. Vá em **Settings**
3. Clique em **Apps / GitHub Apps**
4. Procure por **Reviewable**
5. Clique em **Install**
6. Escolha: apenas um repositório, ou todos os repositórios

3. Passo a passo de uso (com orientação de capturas)

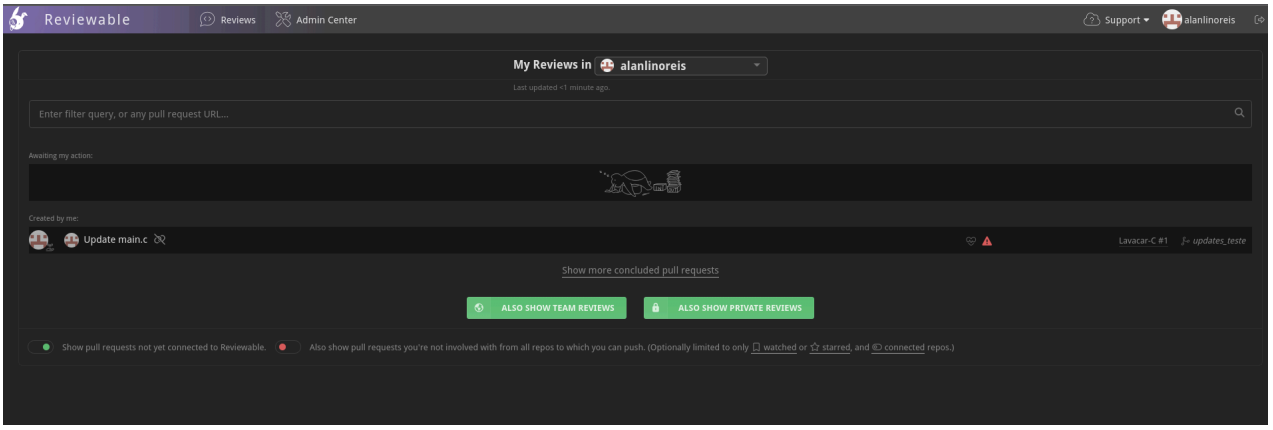
Passo 1: Criar um Pull Request

1. Faça alterações no código localmente
2. Faça commit:
3. No GitHub, clique em **Create pull request**



Passo 2: Abrir no Reviewable

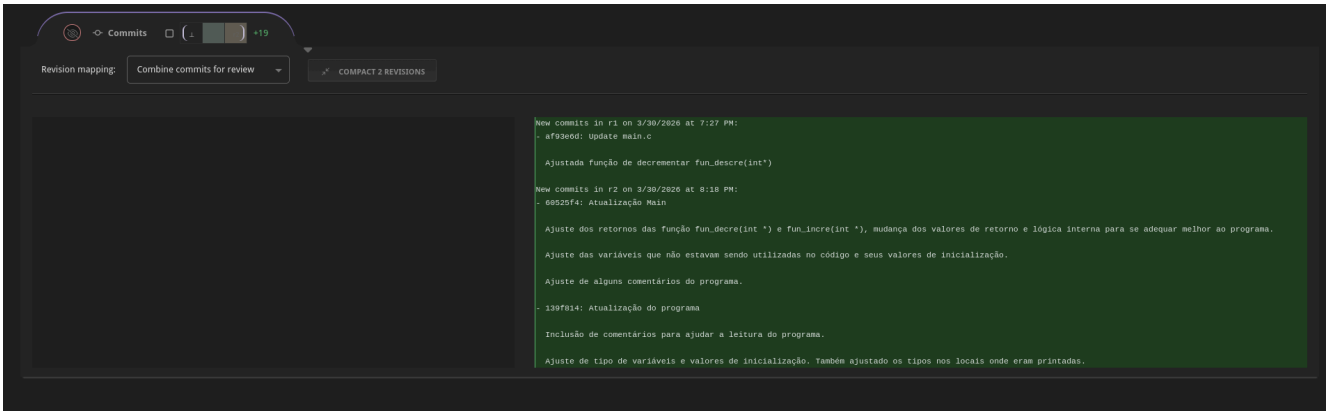
1. Dentro do Pull Request, clique na alteração feita pelo usuário.

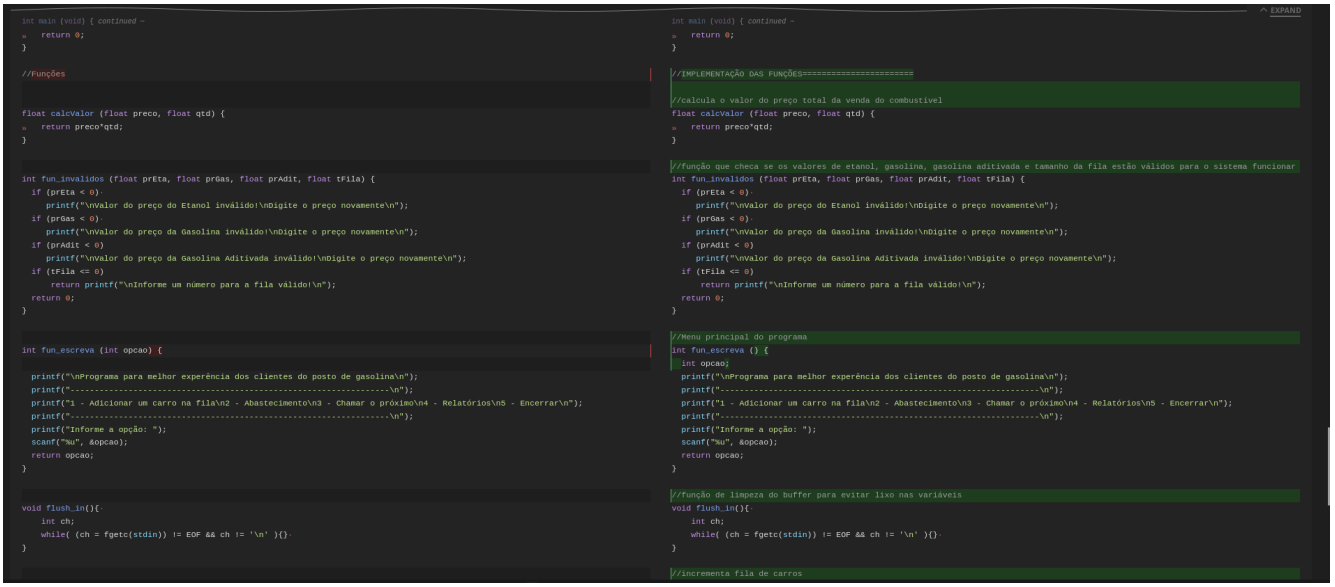


Passo 3: Analisar o código

Na interface do Reviewable:

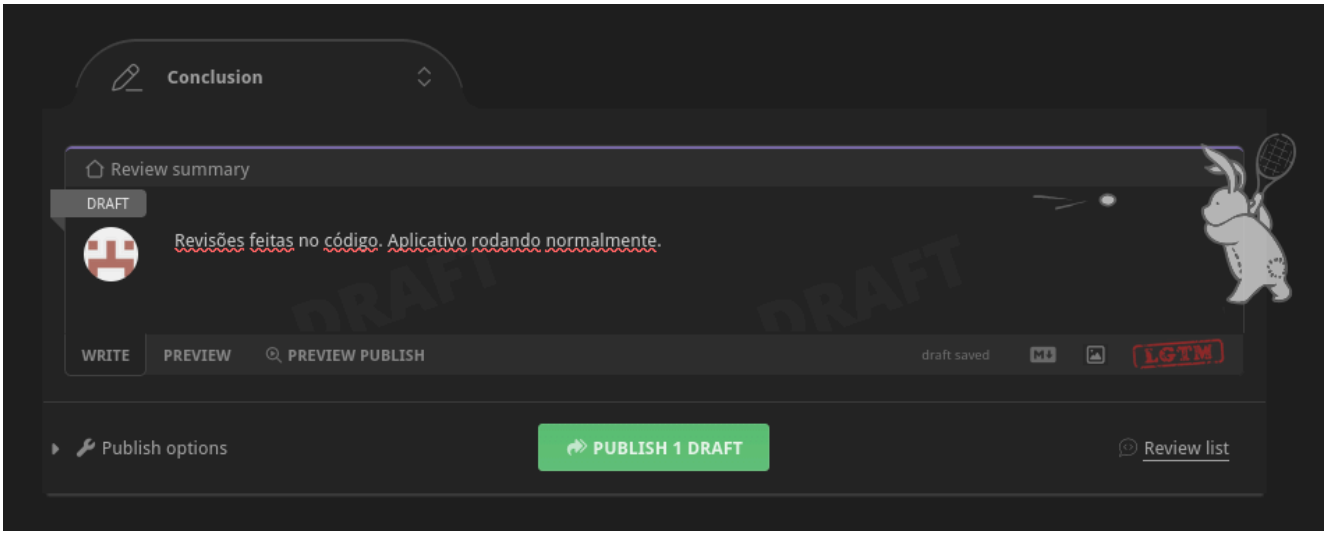
- Veja mudanças linha por linha
- Arquivos são organizados automaticamente
- Comentários aparecem ao lado do código





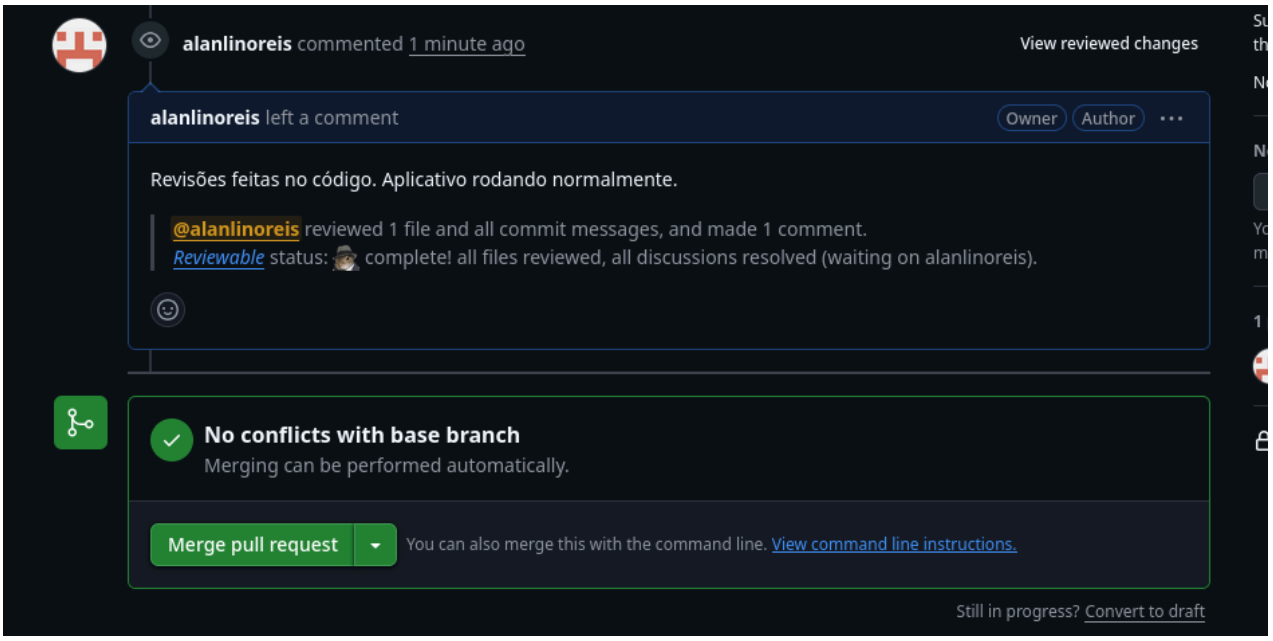
Passo 4: Comentar a revisão

1. Escreva um comentário
2. Clique em **Publish**



Passo 5: Marcar revisão como concluída

- Após revisar tudo, volte ao github na aba dos pull requests e caso as alterações forem aceitas, clique no botão "merge pull request".



4. Demonstração de revisão no código

1°Alteração: Ajuste da estrutura TCarro, ajuste dos parâmetros de entrada e saída das funções e ajuste das variáveis do sistema.

```c
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

//Struct para os carros
struct Tcarro{
  char modelo[30] ;
  char placa[30] ;
  char cor[30];
};

//Funções
int fun_decre();
int fun_incre(int *);
void flush_in ();
float calcValor (float preco, float qtd);
int fun_invalidos (float prEta, float prGas, float prAdit, float tFila);
int fun_escreva();
char Fun_Tcar();

//MAIN
int main (void) {

  //VARIÁVEIS
  float  prEta = 1, prGas = 1, prAdit = 1, qtdEta, qtdGas, qtdAdit, valRec, cAtendido, tFila = 1;
  float etaRestante = 200, gasRestante = 200, aditRestante = 200;
  int fila = 0, opcao, relatorio, menu, tam, cont = 1;

  printf("-----------------------------------------------------------------------------------------------------\n");
  printf("Bem vindo ao programa de abastecimento do posto de gasolina! Nosso programa facilita o abastecimento para nossos cliente
s.\n\n                    \n");
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

//Struct para os carros
typedef struct Tcarro{
  char modelo[30] ;
  char placa[30] ;
  char cor[30];
}Tcarro;

//Definição do escopo das funções para implementação
void fun_decre(int *);
void fun_incre(int *);
void flush_in ();
float calcValor (float preco, float qtd);
int fun_invalidos (float prEta, float prGas, float prAdit, float tFila);
int fun_escreva();
char Fun_Tcar();

//MAIN
int main (void) {

  //VARIÁVEIS
  float  prEta, prGas, prAdit, qtdEta, qtdGas, qtdAdit, valRec;
  float etaRestante = 200, gasRestante = 200, aditRestante = 200;
  int fila = 0, tFila = 1, relatorio, menu, cont = 0, cAtendido = 0;

  printf("-----------------------------------------------------------------------------------------------------\n");
  printf("Bem vindo ao programa de abastecimento do posto de gasolina! Nosso programa facilita o abastecimento para nossos cliente
s.\n\n                    \n");
```

COLLAPSED: 12 lines with no changes.

EXPAND

2°Alteração: Comentário: Alteração no parâmetro de entrada do tamanho da fila, ajuste da comparação da fila com o tamanho da fila.

```c
int main (void) { continued ...
  scanf  ("%f", &prAdit);

  printf("\nInforme o tamanho da fila que o estabelecimento suportará: ");
  scanf  ("%2f", &tFila);

  }
  while (prEta < 0 || prGas < 0 || prAdit < 0 || tFila <= 0);

  struct Tcarro carros [tam];
  do {

  menu = fun_escreva ();

  switch (menu){
    case 1:
      system("clear");
      if (fila <= tFila) {
        fun_incre(&fila);
        flush_in ();
........

        printf("\nDigite o modelo do carro:\n ");
        fgets(carros[cont].modelo,30,stdin);
```

```c
int main (void) { continued ...
  scanf  ("%f", &prAdit);

  printf("\nInforme o tamanho da fila que o estabelecimento suportará: ");
  scanf  ("%d", &tFila);

  }
  while (prEta < 0 || prGas < 0 || prAdit < 0 || tFila <= 0);

  Tcarro carros[tFila];
  do {

  menu = fun_escreva();

  switch (menu){
    case 1:
      system("clear");
      if (fila < tFila) {
        fun_incre(&fila);
        flush_in ();

........
        printf("\nDigite o modelo do carro:\n ");
        fgets(carros[cont].modelo,30,stdin);
```

3°Alteração: Adicionado uma na cláusula do switch case para retorno do menu.

COLLAPSED: 32 lines with no changes.

```
int main (void) { continued ⇢
........
        case 3:
          system("clear");system("clear");
          printf ("%.f carros foram atendidos\n", cAtendido);
          break;
........
        case 4:
          system("clear");
          printf ("%.2f litros restante de Etanol\n %.2f litros restante de Gasolina Comum\n %.2f litros restante de Gasolina Adit
ivada", etaRestante, gasRestante, aditRestante);

          break;
........
        default:
```

```
int main (void) { continued ⇢
........
        case 3:
          system("clear");system("clear");
          printf ("%d carros foram atendidos\n", cAtendido);
          break;
........
        case 4:
          system("clear");
          printf ("%.2f litros restante de Etanol\n %.2f litros restante de Gasolina Comum\n %.2f litros restante de Gasolina Adit
ivada", etaRestante, gasRestante, aditRestante);
          break;
........
        case 5:
          system("clear");
          break;

        default:
```

EXPAND

COLLAPSED: 16 lines with no changes.

EXPAND

4° Alteração: Adicionando comentários para melhor legibilidade do sistema.

```c
int main (void) { continued ⋯
    return 0;
}

//Funções


float calcValor (float preco, float qtd) {
    return preco*qtd;
}


int fun_invalidos (float prEta, float prGas, float prAdit, float tFila) {
  if (prEta < 0)
     printf("\nValor do preço do Etanol inválido!\nDigite o preço novamente\n");
  if (prGas < 0)
     printf("\nValor do preço da Gasolina inválido!\nDigite o preço novamente\n");
  if (prAdit < 0)
     printf("\nValor do preço da Gasolina Aditivada inválido!\nDigite o preço novamente\n");
  if (tFila <= 0)
      return printf("\nInforme um número para a fila válido!\n");
  return 0;
}
```

```c
int main (void) { continued ⋯
    return 0;
}

//IMPLEMENTAÇÃO DAS FUNÇÕES=======================

//calcula o valor do preço total da venda do combustivel
float calcValor (float preco, float qtd) {
    return preco*qtd;
}

//função que checa se os valores de etanol, gasolina, gasolina aditivada e tamanho da fila estão válidos para o sistema funcionar
int fun_invalidos (float prEta, float prGas, float prAdit, float tFila) {
  if (prEta < 0)
     printf("\nValor do preço do Etanol inválido!\nDigite o preço novamente\n");
  if (prGas < 0)
     printf("\nValor do preço da Gasolina inválido!\nDigite o preço novamente\n");
  if (prAdit < 0)
     printf("\nValor do preço da Gasolina Aditivada inválido!\nDigite o preço novamente\n");
  if (tFila <= 0)
      return printf("\nInforme um número para a fila válido!\n");
  return 0;
}
```

5°Alteração: Adicionado comentários, valores de retorno das funções e criação de variáveis auxiliares.

```c
int fun_escreva (int opcao) {

  printf("\nPrograma para melhor experência dos clientes do posto de gasolina\n");
  printf("-----------------------------------------------------------------\n");
  printf("1 - Adicionar um carro na fila\n2 - Abastecimento\n3 - Chamar o próximo\n4 - Relatórios\n5 - Encerrar\n");
  printf("-----------------------------------------------------------------\n");
  printf("Informe a opção: ");
  scanf("%u", &opcao);
  return opcao;
}

void flush_in(){
    int ch;
    while( (ch = fgetc(stdin)) != EOF && ch != '\n' ){}
}

int fun_incre (int *fila){
  *fila = *fila + 1;
  printf ("Número de carros na fila: %d\n", *fila);
 return *fila;
    }

int fun_decre (int *fila){
  *fila = *fila - 1;
  printf ("Número de carros na fila: %d\n", *fila);
 return *fila;
 }
```

```c
//Menu principal do programa
int fun_escreva () {
  int opcao;
  printf("\nPrograma para melhor experência dos clientes do posto de gasolina\n");
  printf("-----------------------------------------------------------------\n");
  printf("1 - Adicionar um carro na fila\n2 - Abastecimento\n3 - Chamar o próximo\n4 - Relatórios\n5 - Encerrar\n");
  printf("-----------------------------------------------------------------\n");
  printf("Informe a opção: ");
  scanf("%u", &opcao);
  return opcao;
}

//função de limpeza do buffer para evitar lixo nas variáveis
void flush_in(){
    int ch;
    while( (ch = fgetc(stdin)) != EOF && ch != '\n' ){}
}

//incrementa fila de carros
void fun_incre (int *fila){
  *fila = *fila + 1;
  printf ("Número de carros na fila: %d\n", *fila);

}

//decrementa fila de carros
void fun_decre (int *fila){
  *fila = *fila - 1;
  printf ("Número de carros na fila: %d\n", *fila);

 }
```

6°Alteração: Adicionados comentários no código referentes a opções de atualização do software e remoção de comentários redundantes (que não traziam nenhuma diferença para o entendimento).

```c
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

//Struct para os carros
typedef struct Tcarro{
  char modelo[30] ;
  char placa[30] ;
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

typedef struct Tcarro{
  char modelo[30] ;
  char placa[30] ;
```

```c
int fun_escreva();
char Fun_Tcar();

//MAIN
int main (void) {

  //VARIÁVEIS
  float  prEta, prGas, prAdit, qtdEta, qtdGas, qtdAdit, valRec;



  float etaRestante = 200, gasRestante = 200, aditRestante = 200;
  int fila = 0, tFila = 1, relatorio, menu, cont = 0, cAtendido = 0;

  printf("-----------------------------------------------------------------------------------
---------------\n");
  printf("Bem vindo ao programa de abastecimento do posto de gasolina! Nosso programa facilita o aba
stecimento para nossos clientes.\n\n                    \n");
  printf("-----------------------------------------------------------------------------------
---------------\n");
..
  do {



  fun_invalidos(prEta, prGas, prAdit, tFila);
....
  printf ("\nDigite o preço do etanol: ");
```

```c
int fun_escreva();
char Fun_Tcar();

int main (void) {

  //Quantidade de variáveis em excesso, algumas poderiam ser criadas apenas no escopo da função
  float  prEta, prGas, prAdit, qtdEta, qtdGas, qtdAdit, valRec;
  //Opção de melhoria, inserir manualmente a quantidade de combustivel ou possivel integração com um
 sistema de gerenciamento de quantidade

  float etaRestante = 200, gasRestante = 200, aditRestante = 200;
  int fila = 0, tFila = 1, relatorio, menu, cont = 0, cAtendido = 0;

  printf("-----------------------------------------------------------------------------------
---------------\n");
  printf("Bem vindo ao programa de abastecimento do posto de gasolina! Nosso programa facilita o aba
stecimento para nossos clientes.\n\n                    \n");
  printf("-----------------------------------------------------------------------------------
---------------\n");
..
  do {
    // dentro fun_invalidos deveria-se checar se a a atribuição de valores correspondentes ao seu ti
po, já que dentro do loop já é checado se os valores são negativos ou válidos
    // deveria-se identificar os valores individualmente, para que não seja necessário digitar a mes
ma informação mais de uma vez mesmo que ela esteja correta
  fun_invalidos(prEta, prGas, prAdit, tFila);
....
  printf ("\nDigite o preço do etanol: ");
```

```c
int main (void) { continued ...
      }
    break;
....

    case 2:
      system("clear");
```

```c
int main (void) { continued ...
      }
    break;
....
    // Opção para atualizações futuras, poder selecionar um tipo de combustivel por vez
    case 2:
      system("clear");
```

5. Observações sobre a experiência

Pontos positivos

- Interface limpa e organizada;
- Excelente para revisões detalhadas;
- Histórico de revisão bem estruturado;
- Integração direta com GitHub.

Pontos negativos

- Pode ser confuso no início;
- Algumas funções são menos intuitivas;
- Dependência do GitHub (não funciona standalone).